

RICARDO LEOPOLDO TARACENA GONZALEZ
07192202

PILA LIFO

En esta implementación, la pila se construyó de forma dinámica mediante una lista simplemente enlazada de nodos. Cada nodo contiene un valor de tipo texto (data) y una referencia o puntero al siguiente elemento (next).

```
2 public class main{
3     static class Node{
4         String data;
5         Node next;
6
7         Node(String data){
8             this.data=data;
9             this.next=null;
10        }
11    }
12    static class Stack{
13        Node top;
14
15        Stack(){
16            top=null;
17        }
18    }
19 }
```

TOP

pertenece a la estructura Stack y mantiene la referencia directa al nodo situado en la cima de la pila. Actúa como el único punto de entrada y salida de datos.

```
static class Stack{
    Node top;

    Stack(){
        top=null;
    }
}
```

PUSH

Crea un nuevo nodo, asigna su referencia next hacia el top actual y actualiza top para que apunte a este nuevo nodo.

```
public void push(String data){  
    Node newNode=new Node(data);  
    newNode.next=top;  
    top=newNode;  
}
```

POP

Recupera el valor del nodo en top y desplaza top al nodo consecutivo (top = top.next), removiendo el elemento superior.

```
public String pop(){  
    if(top==null){  
        return "la pila esta vacia";  
    }  
  
    String value=top.data;  
    top=top.next;  
    return value;  
}
```

PEEK

Devuelve el valor contenido en top para inspeccionar el elemento superior sin modificar la estructura.

```
public String peek(){  
    if(top==null){  
        return "la pila esta vacia";  
    }  
  
    return top.data;  
}
```

COLA (FIFO)

El primer elemento en ingresar es el primero en ser procesado. Al igual que la pila, se implementó de forma dinámica utilizando nodos enlazados.

```
public class queue{
    static class Node{
        String data;
        Node next;

        Node(String data){
            this.data=data;
            this.next=null;
        }
    }
    static class Queue{
        Node front;
        Node rear;

        Queue(){
            front=null;
            rear=null;
        }
    }
}
```

FRONT

Apunta al primer nodo de la cola, es decir, al elemento que será atendido o retirado a continuación.

REAR

Apunta al último nodo insertado, asegurando que los nuevos elementos se añadan al final

```
static class Queue{  
    Node front;  
    Node rear;  
  
    Queue(){  
        front=null;  
        rear=null;  
    }  
}
```

ENQUEUE

Crea un nuevo nodo. Si la cola está vacía, define a dicho nodo tanto como front como rear. Si ya contiene elementos, enlaza el rear actual con el nuevo nodo y actualiza rear a esta nueva posición.

```
public void enqueue(String data){  
    Node newNode=new Node(data);  
    if(front==null ){  
        front=newNode;  
        rear=newNode;  
        return;  
    }  
  
    rear.next=newNode;  
    rear=newNode;  
}
```

DEQUEUE

Extrae el valor del nodo en front y mueve front al siguiente nodo (front = front.next). Si al retirar el elemento la cola queda vacía (front == null), reinicia rear a null.

```

public String dequeue(){
    if (front==null){
        return"la cola esta vacia";
    }

    String value=front.data;
    front=front.next;

    if (front==null){
        rear=null;
    }

    return value;
}

```

PEEK

Retorna el valor en front para consultar la siguiente tarea a procesar sin alterar la cola.

```

public String peek(){
    if (front==null){
        return "la cola esta vacia";
    }
    return front.data;
}

```

ESTRUCTURAS VACIAS

Validación previa: Todas las operaciones de consulta o extracción (pop, dequeue, peek, Historial, Pendientes) verifican primero si la estructura está vacía validando si `top == null` o `front == null`.

```
public String dequeue(){  
    if (front==null){  
        return "la cola esta vacia";  
    }  
}
```

Respuestas controladas: En caso de operar sobre una estructura vacía, las funciones retornan un mensaje descriptivo de texto (por ejemplo, "La pila está vacía" o "La cola está vacía") en lugar de interrumpir la ejecución del programa.

```
public String pop(){  
    if(top==null){  
        return "la pila esta vacia";  
    }  
}
```

PRUEBAS

COLA

```

public static void main(String[] args) {
    Queue queue = new Queue();
    //COLA VACIA
    System.out.println(queue.dequeue());
    System.out.println(queue.size());
    System.out.println(queue.peek());
    System.out.println(queue.isEmpty());
    //PRUEBA
    queue.enqueue(data: "tarea 1");
    queue.enqueue(data: "tarea 2");
    queue.enqueue(data: "tarea 3");

    System.out.println(queue.peek());

    System.out.println(queue.dequeue());
    System.out.println(queue.peek());
    System.out.println(queue.size());
    System.out.println(queue.isEmpty());

    System.out.println(queue.Pendientes());
}
}

```

RESULTADO

```

la cola esta vacia
0
la cola esta vacia
true
tarea 1
tarea 1
tarea 2
2
false
tarea 2 <- FRONT
tarea 3 <- REAR

```

STACK

```

Run main | Debug main | Run | Debug
public static void main(String[] args) {
    Stack stack=new Stack();
    //STACK VACIO
    System.out.println(stack.peek());
    System.out.println(stack.size());
    System.out.println(stack.pop());

    //PRUEBA
    stack.push(data: "crear documento");
    stack.push(data: "editar documento");
    stack.push(data: "renombrar documento");

    System.out.println(stack.peek());
    System.out.println(stack.pop());
    System.out.println(stack.peek());
    System.out.println(stack.size());
    System.out.println(stack.isEmpty());
}

```

RESULTADO

```

la pila esta vacia
0
la pila esta vacia
renombrar documento
renombrar documento
editar documento
2
false

```

agregar acciones


```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 1
registra la accion
Accion 1
```

consultar la última acción

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 3
Ultima accion
Accion 1
```

deshacer acciones

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 2
Ultima accion
Accion 1
eliminada con exito
```

mostrar historial

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 4
historial
La pila está vacía
```

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 1
registra la accion
Accion 1
```

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 1
registra la accion
accion 2
```

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 4
historial
accion 2 <- TOP
Accion 1
```

agregar tareas

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 5
ingrese la tarea
TAREA 1
Agregado
```

consultar la siguiente tarea

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 7
procesando...
TAREA 1
```

procesar tareas

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 6
procesando...
TAREA 1
```

mostrar tareas pendientes

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 8
Tareas pendientes:
La cola está vacía
```

=====

CENTRO DE OPERACIONES

=====

1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 5
ingrese la tarea
TAREA 1
Agregado

=====

CENTRO DE OPERACIONES

=====

1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 5
ingrese la tarea
TAREA 2
Agregado

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 8
Tareas pendientes:
TAREA 1 <- FRONT
TAREA 2 <- REAR
```

manejo de Stack vacía

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 2
Ultima accion
la pila esta vacia
eliminada con exito
```


manejo de Queue vacía.

```
=====
CENTRO DE OPERACIONES
=====
1. Insertar accion
2. Eliminar accion
3. Ver ultima accion
4. Mostrar Historial
5. Ingresar tarea
6. Procesar tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes
9. Estado del sistema
10. Salir
Selecciona una opción (1-10): 6
procesando...
la cola esta vacia
```

COMPORTAMIENTO DE LA PILA VS COLA PILA

```
Stack stack=new Stack();
System.out.println(x: "AGREGANDO A,B,C,D");
stack.push(data: "A");
System.out.println(stack.peek());
stack.push(data: "B");
System.out.println(stack.peek());
stack.push(data: "C");
System.out.println(stack.peek());
stack.push(data: "D");
System.out.println(stack.peek());

System.out.println(x: "ELIMINANDO D,C,B,A");

System.out.println(stack.pop());
System.out.println(stack.pop());
System.out.println(stack.pop());
System.out.println(stack.pop());
```

```
AGREGANDO A,B,C,D  
A  
B  
C  
D  
ELIMINANDO D,C,B,A  
D  
C  
B  
A
```

COLA

```
Queue queue =new Queue();  
System.out.println(x: "AGREGANDO A,B,C,D");  
queue.enqueue(data: "A");  
queue.enqueue(data: "B");  
queue.enqueue(data: "C");  
queue.enqueue(data: "D");  
System.out.println(queue.Pendientes());  
  
System.out.println(x: "ELIMINANDO A,B,C,D");  
System.out.println(queue.dequeue());  
System.out.println(queue.dequeue());  
System.out.println(queue.dequeue());  
System.out.println(queue.dequeue());
```

AGREGANDO A,B,C,D

A <- FRONT

B

C

D <- REAR

ELIMINANDO A,B,C,D

A

B

C

D